MVC — Views (Observers) & Controllers
«View» «Observer»
DonationPanel
-donorId: Integer (null = admin)
+databaseChanged(): void
+refresh(): void
«View» «Observer»
BeneficiaryPanel
-me: Integer (null = admin)
+databaseChanged(): void
+refresh(): void
«View» «Observer»
VolunteerPanel
-volunteerId: Integer (null = admin)
+databaseChanged(): void
+refresh(): void
+showMessage(text) / showError(text)
«View» «Observer»
EventPanel
+databaseChanged(): void
+refresh(): void
«View»
CommunicationPanel
+refresh(): void
calls
calls
updates
calls
calls
«Controller»
DonationController
-donationService: DonationService
+addDonor(...) / updateDonor(...) / deleteDonor(id)
+recordDonation(donorId, amount, strategy): Donation
+generateReceipt(id) / generateTaxSummary(id, year)
«Controller»
BeneficiaryController
-service: BeneficiaryService
+register(...) / update(...) / deleteBeneficiary(id)
+requestResource(...) / distribute(...)
+addResource(...) / restock(...)
+crudMatrix(): List<Row>
«Controller»
VolunteerController
-service: VolunteerService
-currentUser: User
-view: VolunteerPanel
+handleRegister(...) / handleUpdateProfile(...)
+handleCreateTask(type, ...)
+handleAssignTask(v, task, role)
+handleLogHours(...) / handleApproveHours(...)
+handleIssueCertificate(v, type)
«Controller»
EventController
-service: EventService
+createEvent(...) / updateEvent(...) / cancelEvent(id)
+registerAttendee(eventId, userId)
+checkIn(ticketCode)
+sendReminder(eventId)
«Controller»
CommunicationController
-service: NotificationService
+sendAnnouncement(group, subject, text)
+listHistory(): List<Message>
delegates
delegates
delegates
delegates
delegates
Beneficiary Management (CRUD Matrix)
«Service»
BeneficiaryService
-repository: JsonRepository
+register(...) / update(...) / deleteBeneficiary(id)
+addNeeds(...) / requestResource(...) / requestService(...)
+addResource(...) / restock(id, qty) / deleteResource(id)
+distribute(...): ResourceDistribution
+recordService(...) / recordImpact(...)
+history(id) / impactSummary(id): String
manages
provides
Beneficiary
-id: int
-name: String
-contact: String
-address: String
-status: Status
«CRUD look-up table»
CrudMatrix
+ENTITIES: String[]
-ROWS: List<Row>
+NOTES: String
+rows(): List<Row>
receives
1
1
11
1
0..*
0..*
0..*
0..*
0..*
0..*
0..*
ServiceRecord
-id, beneficiaryId, requestId: int
-serviceName: String
ImpactRecord
-id, beneficiaryId: int
-outcome: String
NeedAssessment
-id, beneficiaryId: int
-resourceNeeds: String
-serviceNeeds: String
ResourceDistribution
-id, beneficiaryId, resourceId: int
-quantity: int
-requestId: int
AssistanceRequest
-id, beneficiaryId, resourceId: int
-type: Type (RESOURCE / SERVICE)
-quantity, fulfilledQuantity: int
-status: Status
of
for
adds donated stock
uses
uses
uses
1
0..1
Resource
-id: int
-name: String
-category: String
-stock: int
Donation Management (Strategy + Decorator)
Donor
-donorId: int
-name: String
-email: String
-phone: String
-taxId: String
«Service»
DonationService
-repository: JsonRepository
+addDonor(...) / updateDonor(...) / deleteDonor(id)
+recordDonation(donorId, amount, strategy): Donation
+deleteDonation(id)
+generateReceipt(id): String
+generateTaxSummary(donorId, year): String
creates
1
makes
0..*
«Context»
Donation
-donationId: int
-donorId: int
-amount: double
-donationDate: Date
-status: String
-paymentStrategy: IPaymentStrategy
+setPaymentStrategy(s: IPaymentStrategy)
+processDonation(): boolean
uses
1
«interface» «Strategy»
IPaymentStrategy
+ProcessPayment(amount: double): boolean
+getMethodName(): String
1
CreditCardPayment
-transactionId: String
+ProcessPayment(amount): boolean
DebitCardPayment
-transactionId: String
+ProcessPayment(amount): boolean
wraps
CashPayment
+ProcessPayment(amount): boolean
InKindPayment
-itemName: String
-itemCategory: String
-unitQuantity: int
+ProcessPayment(value): boolean
CheckPayment
-checkNumber: String
-bankName: String
+ProcessPayment(amount): boolean
«abstract» «Decorator»
PaymentDecorator
#wrapped: IPaymentStrategy
+ProcessPayment(amount): boolean
+getMethodName(): String
«Concrete Decorator»
FeeCoveredPayment
+FEE_RATE: double = 0.03
+ProcessPayment(amount): boolean
+getMethodName(): String
Database (Singleton + Observer subject)
«Singleton» «Subject»
JsonRepository
-instance: JsonRepository
-observers: List<DatabaseObserver>
-filePath: Path
-database: Database
-JsonRepository(path)
+getInstance(): JsonRepository
+read(op): T
+update(op): T
+registerObserver(o) / removeObserver(o)
+notifyObservers()
observers
0..*
1
1
«interface» «Observer»
DatabaseObserver
+databaseChanged(): void
Database
+nextUserId, nextId, nextDonationId: int
+donors, donations: Map
+beneficiaries, needs, requests, resources, distributions, ...: Map
+volunteers, volunteerTasks, volunteerAssignments, ...: Map
Volunteer Management (Factory + Repository)
«Service»
VolunteerService
-repository: VolunteerRepository
-taskFactory: VolunteerTaskFactory
-notifier: VolunteerNotifier
+registerVolunteer(...) / updateVolunteer(...)
+createTask(type, title, start, end): VolunteerTask
+findSuitableVolunteers(taskId): List<Volunteer>
+assignTask(volunteerId, taskId, role, by)
+recordHours(...) / approveHours(...)
+issueCertificate(volunteerId, type)
uses
uses
uses
uses
«Factory»
VolunteerTaskFactory
+createTask(type: TaskType, title, start, end): VolunteerTask
«interface» «Repository»
VolunteerRepository
+save(v) / findById(id) / findAll()
+saveTask(t) / findAllTasks()
+saveAssignment(a) / saveHourRecord(r)
+saveCertificate(c)
«interface»
VolunteerNotifier
+notify(recipient: Volunteer, subject, message)
0..*
creates
«abstract»
VolunteerTask
#taskId: int
#title: String
#startTime, endTime: LocalDateTime
#requiredSkills: List<String>
#maxVolunteers: int
#status: TaskStatus
+overlaps(other): boolean
+isFull(): boolean
+getType(): TaskType
+execute(): void
+getTypeDetails(): String
ConsoleVolunteerNotifier
+notify(recipient, subject, message)
JsonVolunteerRepository
-db: JsonRepository
(stores everything in the shared Database)
1
OutreachTask
-targetArea: String
-communityGroup: String
+execute(): void
FundraiserTask
-targetAmount: double
+execute(): void
WorkshopTask
-topic: String
-materials: List<String>
+execute(): void
0..*
VolunteerAssignment
-assignmentId: int
-volunteerId, taskId: int
-role: String
-status: AssignmentStatus
-assignedById: int
+accept() / decline()
+complete() / cancel()
0..*
1
0..*
VolunteerHourRecord
-recordId: int
-assignmentId: int
-date: LocalDate
-hours: double
-approvalStatus: ApprovalStatus
+approve(approver: User)
+reject(approver: User)
uses
getInstance() / registerObserver()
1
Volunteer
-preferences: List<String>
-status: VolunteerStatus
-skills: List<Skill>
-availability: List<AvailabilitySlot>
-approvedHours: double
+addSkill(s) / removeSkill(id)
+hasSkill(name): boolean
+addAvailability(slot)
+isAvailable(start, end): boolean
1
0..*
1
1
earns
0..*
0..*
AvailabilitySlot
-slotId: int
-day: DayOfWeek
-startTime, endTime: LocalTime
+covers(start, end): boolean
VolunteerCertificate
-certificateId: int
-type: CertificateType
-totalHours: double
-completedTasks: int
+generate(): String
Skill
-skillId: int
-name: String
-level: SkillLevel
-certified: boolean
+matches(name): boolean
Communication Module (Strategy) — designed, coded in the next phase
«Service» «Context»
NotificationService
-repository: JsonRepository
approved by
+send(to: User, subject, body)
+sendToGroup(role, subject, body)
-chooseChannel(to: User): ICommunicationChannel
uses
saves
1
«interface» «Strategy»
ICommunicationChannel
+deliver(message: Message): boolean
Message
-messageId: int
-recipientId: int
-subject: String
-body: String
-channel: String
-sentAt: LocalDateTime
SmsChannel
+deliver(message): boolean
SocialMediaChannel
-platform: String
+deliver(message): boolean (public post)
EmailChannel
+deliver(message): boolean
Application & Users (login, one window per role)
Main
-ADMIN_PASSWORD: String
+main(args)
-showLogin()
-openWindow(role, id, who)
-observe(window, screens)
«abstract»
User
#userId: int
#name: String
#email: String
#phone: String
+updateProfile(name, email, phone)
creates admin
EventCoordinator
(admin / coordinator user)
1
registers
uses
uses
Event Management (Observer) — designed, coded in the next phase
«Service»
EventService
-repository: JsonRepository
+createEvent(...) / updateEvent(...) / cancelEvent(id)
+register(eventId, userId): EventRegistration
+checkIn(ticketCode)
+sendReminder(eventId)
+attendanceReport(eventId): String
«interface» «Subject»
EventSubject
+attach(o: EventObserver)
+detach(o: EventObserver)
+notifyObservers(change: String)
manages
«Concrete Subject»
Event
-eventId: int
-title: String
-type: EventType (FUNDRAISER / WORKSHOP / OUTREACH)
-dateTime: LocalDateTime
-location: String
-capacity: int
-status: EventStatus
-observers: List<EventObserver>
+attach(o) / detach(o)
+notifyObservers(change)
+hasCapacity(): boolean
+updateDetails(...)
+cancel()
staffed by
0..1
observers
0..*
1
«interface» «Observer»
EventObserver
+update(event: Event, change: String)
«Concrete Observer»
AttendeeNotifier
+update(event, change) (messages registered attendees)
«Concrete Observer»
VolunteerEventNotifier
+update(event, change) (messages assigned volunteers)
0..*
0..*
EventRegistration
-registrationId: int
-eventId: int
-userId: int
-status: RegistrationStatus
+confirm()
+cancel()
1
1
0..1
EventTicket
-ticketCode: String
-status: TicketStatus (VALID / USED / VOID)
+validate(): boolean
AttendanceRecord
-attended: boolean
-checkInTime: LocalDateTime
+markPresent()